



DECISION MAKING

BEHAVIOUR TREES

Jin Weijian Victor Chen - 100496723

EXERCISE 1

Changing the type of composite node to a selector was enough to ensure the correct execution of the tree.

EXERCISE 2

code saved in the zipfile. Two different outputs below:

```
backtoaudio@kitty:~/decision_making$ /bin/python3 /home/backtoaudio/decision_making/decision_making.py
LocationCheck behavior initialized
MoveToLocation behavior initialized
PickUpObject behavior initialized

Checking current location...
Current location: 5, Target location: 5
Already at the target location
Attempting to pick up the object...
Successfully picked up the object
{-} Pick Object Sequence [o]
  {o} Move Sequence [o]
    --> Check Location [o]
    --> Move to Location [-]
    --> Pick Up Object [o]

backtoaudio@kitty:~/decision_making$ /bin/python3 /home/backtoaudio/decision_making/decision_making.py
LocationCheck behavior initialized
MoveToLocation behavior initialized
PickUpObject behavior initialized

Checking current location...
Current location: 6, Target location: 5
Not at the target location
Moving to location 5...
Failed to move to the target location
{-} Pick Object Sequence [x]
  {o} Move Sequence [x]
    --> Check Location [x]
    --> Move to Location [x]
    --> Pick Up Object [-]

backtoaudio@kitty:~/decision_making$
```

EXERCISE 3

```
--- New Tick 4 ---
Checking current location...
Current location: 0, Target location: 5
Not at the target location
Moving to location 5...
Failed to move to the target location
{-} Pick Object Sequence [x]
  {o} Move Sequence [x]
    --> Check Location [x]
    --> Move to Location [x]
    --> Pick Up Object [-]

--- New Tick 5 ---
Checking current location...
Current location: 4, Target location: 5
Not at the target location
Moving to location 5...
Successfully moved to the target location
Attempting to pick up the object...
Failed to pick up the object
{-} Pick Object Sequence [x]
  {o} Move Sequence [o]
    --> Check Location [x]
    --> Move to Location [o]
    --> Pick Up Object [x]
```

EXERCISE 4

In the original code, 3 behaviors (check battery, check obstacle and navigate) were done simultaneously, under the same parallel node, but for safe navigation we need to wait until both battery and obstacle returns successfully, so i changed the root to be a sequential, and only when the parallel node containing battery and obstacle returns both true, moving forward will be allowed.

```
backtoaudio@kitty:~/decision_making$ /bin/python3 /h
No obstacle detected.
Battery level ok: 81%
Navigating safely...
{-} move forward [o]
  /_ Parallel: Obstacle and Battery Check [o]
    --> Check For Obstacles [o]
    --> Check Battery Level [o]
    --> Move Forward [o]

backtoaudio@kitty:~/decision_making$ /bin/python3 /h
No obstacle detected.
Battery level too low: 17%
{-} move forward [x]
  /_ Parallel: Obstacle and Battery Check [x]
    --> Check For Obstacles [o]
    --> Check Battery Level [x]
    --> Move Forward [-]
```

EXERCISE 5

The output tree will return *, meaning that the process is running, since the visualization occurs during the tick and not after its end.

```
backtoaudio@kitty:~/decision_making$ /bin/pyt
Returning FAILURE, but Inverter will flip it
-^- Succeed after fail [*]
--> Always Fail [*]
```

EXERCISE 6

```
Tick 1
Always failing
-^- three times retry [*] -- attempt failed [status: 1 failure from 3]
--> Always Fail [X]

Tick 2
Always failing
-^- three times retry [*] -- attempt failed [status: 2 failure from 3]
--> Always Fail [X]

Tick 3
Always failing
-^- three times retry [X] -- final failure [status: 3 failure from 3]
--> Always Fail [X]

Tick 4
Always failing
-^- three times retry [*] -- attempt failed [status: 1 failure from 3]
--> Always Fail [X]
```

In the third tick, the decorator returns failure itself, while in the first and second returns running. however in the fourth tick the bt restarts its execution and the decorator returns running again

EXERCISE 7

With `OneShotPolicy.ON_COMPLETION`, it doesn't matter if it returns success or failure once the node is completed it will never run again

```
^Cbacktoaudio@kitty:~/decision_making$ /bin
Message printed only once: SUCCESS
^Cbacktoaudio@kitty:~/decision_making$ /bin
Message printed only once: SUCCESS
^Cbacktoaudio@kitty:~/decision_making$ /bin
Message printed only once: FAILURE
^Cbacktoaudio@kitty:~/decision_making$
```

EXERCISE 8

code in zip file, some examples below.

```
--- Tick 1 ---
Human detected! :)
Hello! Nice to meet you.
How are you today?
Good!!, what a handsome robot you are.
Thanks man, it's been great talking to you.
User needs assistance.
Here's the assistance you needed!
{o} Root [o]
  {-} Interaction Sequence [o]
    ^- Retry Detection [o] -- succeeded [status: 0 failure from 3]
      --> Detect Human [o]
    ^- One Shot Greet [o] -- oneshot completed
      --> Greet [o]
      --> Engage in Conversation [o]
    {-} Assistance Sequence [o]
      --> Need Assistance [o]
      --> Provide Assistance [o]
  --> Idle [-]
```

```

--- Tick 3 ---
Human detected! :)
Hello! Nice to meet you.
How are you today?
Good!!, what a handsome robot you are.
Thanks man, it's been great talking to you.
User doesn't need assistance.
Robot is in idle state.
Robot sleeping 4 seconds
{o} Root [o]
  {-} Interaction Sequence [x]
    ^~ Retry Detection [o] -- succeeded [status: 2 failure from 3]
      --> Detect Human [o]
    ^~ One Shot Greet [o] -- oneshot completed
      --> Greet [o]
    --> Engage in Conversation [o]
  {-} Assistance Sequence [x]
    --> Need Assistance [x]
    --> Provide Assistance [-]
  --> Idle [o]

--- Tick 4 ---
No human detected. :(
{o} Root [*]
  {-} Interaction Sequence [*]
    ^~ Retry Detection [*] -- attempt failed [status: 1 failure from 3]
      --> Detect Human [x]
    ^~ One Shot Greet [-] -- oneshot completed
      --> Greet [-]
    --> Engage in Conversation [-]
  {-} Assistance Sequence [-]
    --> Need Assistance [-]
    --> Provide Assistance [-]
  --> Idle [o]

```

Ctrl+K to generate a command

```

--- Tick 5 ---
No human detected. :(
{o} Root [*]
  {-} Interaction Sequence [*]
    ^~ Retry Detection [*] -- attempt failed [status: 2 failure from 3]
      --> Detect Human [x]
    ^~ One Shot Greet [-] -- oneshot completed
      --> Greet [-]
    --> Engage in Conversation [-]
  {-} Assistance Sequence [-]
    --> Need Assistance [-]
    --> Provide Assistance [-]
  --> Idle [o]

```

